

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет	Программной Инженерии и Компьютерной Техники
Направление подготовки (специальность)	Нейротехнологии и программирование
Дисциплина	Программирование на Python

Лабораторная работа 8
ОТЧЕТ

Выполнил студент:	Иголкин Владислав Андреевич (504623)
Группа:	P3124
Преподаватель:	Жуков Николай Николаевич (261087)

г. Санкт-Петербург
2025 г.

1. Цель работы

1. Создать простое клиент-серверное приложение на Python без серверных фреймворков.
2. Освоить работу с HTTPServer и маршрутизацию запросов.
3. Применять шаблонизатор Jinja2 для отображения данных.
4. Реализовать модели предметной области (User, Currency, UserCurrency, App, Author) с геттерами и сеттерами.
5. Структурировать код в соответствии с архитектурой MVC.
6. Получать данные о курсах валют через функцию `get_currencies` и отображать их пользователям.
7. Реализовать функциональность подписки пользователей на валюты и отображение динамики их изменения.
8. Научиться создавать тесты для моделей и серверной логики.

2. Описание предметной области

Модели:

- Author
 - name — имя автора
 - group — учебная группа
- App
 - name — название приложения
 - version — версия приложения
 - author — объект Author
- User
 - id — уникальный идентификатор
 - name — имя пользователя
- Currency
 - id — уникальный идентификатор
 - num_code — цифровой код
 - char_code — символьный код
 - name — название валюты
 - value — курс
 - nominal — номинал (за сколько единиц валюты указан курс)
- UserCurrency
 - id — уникальный идентификатор
 - user_id — внешний ключ к User
 - currency_id — внешний ключ к Currency
 - реализует связь «много ко многим» между пользователями и валютами.

3. Структура проекта

Мой проект имеет такую структуру:

```
myapp/
├── myapp.py          # главный файл: запускает сервер и показывает страницы
├── models/           # папка с моделями
│   ├── __init__.py
│   ├── author.py     # модель автора
│   ├── app.py        # модель приложения
│   ├── user.py       # модель пользователя
│   ├── currency.py   # модель валюты
│   └── user_currency.py # модель подписки на валюту
└── templates/        # HTML-шаблоны страниц
```

```

├── index.html      # главная страница
├── users.html      # список пользователей
├── currencies.html  # курсы валют
└── utils/
    └── currencies_api.py # функция для получения актуальных курсов валют

```

В myapp.py я настроил сервер и маршрутизацию ссылок.

В models/ хранятся все классы для данных.

В templates/ я сделал шаблоны HTML с переменными Jinja2.

В utils/currencies_api.py — функция для получения курсов валют с сайта ЦБ РФ.

4. Описание реализации

4.1 Реализация моделей

Я сделал все модели с геттерами и сеттерами для проверки данных.

Пример для класса Author:

```

class Author:
    def __init__(self, name, group):
        self.name = name
        self.group = group

    @property
    def name(self):
        return self.__name

    @name.setter
    def name(self, value):
        if isinstance(value, str) and len(value) >= 2:
            self.__name = value
        else:
            raise ValueError('Ошибка при задании имени автора')

```

Я проверяю, что имя и группа — строки и не пустые.

Модели не содержат кода для работы с сервером или HTML.

4.2 Маршруты и обработка запросов

Я использовал стандартный Python-класс BaseHTTPRequestHandler для сервера.

Пример маршрутизации:

```

if path == '/':
    html = template_index.render(
        myapp=app_info.name,
        author_name=main_author.name,
        group=main_author.group,
        navigation=[
            {'caption': 'Главная', 'href': '/'},
            {'caption': 'Пользователи', 'href': '/users'},
            {'caption': 'Валюты', 'href': '/currencies'}

```

```

    ]
)
elif path == '/users':
    html = template_users.render(users=users)
elif path == '/currencies':
    html = template_currencies.render(currencies=currencies)
else:
    html = '<h1>404 - Страница не найдена</h1>'

```

/ — главная страница;
 /users — список пользователей;
 /currencies — список валют;
 всё остальное — 404 страница.

4.3 Использование Jinja2

Для шаблонов я сделал так:

```

from jinja2 import Environment, PackageLoader, select_autoescape

env = Environment(
    loader=PackageLoader('myapp'),
    autoescape=select_autoescape()
)

template_index = env.get_template('index.html')

```

Я инициализировал один раз объект Environment, чтобы Jinja2 кэшировал шаблоны. В шаблоны передаю данные моделей (например, имя автора, список пользователей, курсы валют).

4.4 Интеграция функции get_currencies

Функция берёт курсы валют с сайта Центрального банка РФ:

```

from utils.currencies_api import get_currencies

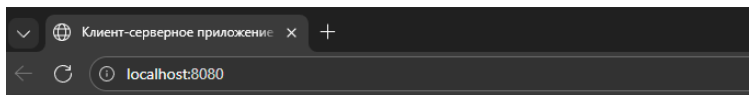
codes = ['USD', 'EUR', 'GBP']
data = get_currencies(codes)

currencies = []
for code, value in data.items():
    currencies.append(Currency(None, None, code, None, value, 1))

```

Я передаю коды валют, получаю словарь с курсами и создаю объекты Currency. Потом эти объекты передаю в шаблон currencies.html.

5. Примеры работы приложения:

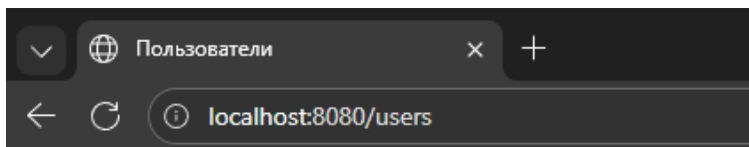


Автор проекта: Владислав Иголкин 504623

Группа: P3124

- [Главная](#)
- [Пользователи](#)
- [Валюты](#)

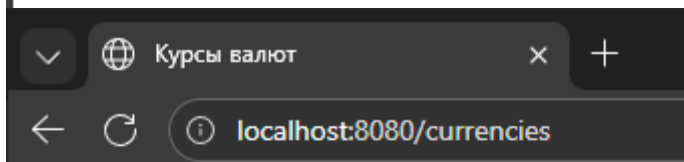
Клиент-серверное приложение на Python



Список пользователей

- Ярослав (ID: 1)
- Виктория (ID: 2)
- Антон (ID: 3)

[На главную](#)



Курсы валют

- USD — 76.0382
- EUR — 88.7898
- GBP — 102.1041

[На главную](#)

7. Тестирование

Пример тестов:

```
import unittest
from jinja2 import Environment, PackageLoader, select_autoescape

from models import Author, App, User, Currency
from utils.currencies_api import get_currencies
```

```

# 1. Тестирование моделей
class TestModels(unittest.TestCase):

    def test_author(self):
        a = Author("Владислав", "P3124")
        self.assertEqual(a.name, "Владислав")
        self.assertEqual(a.group, "P3124")
        with self.assertRaises(ValueError):
            a.name = ""
        with self.assertRaises(ValueError):
            a.group = "123"

    def test_user(self):
        u = User(1, "Ярослав")
        self.assertEqual(u.id, 1)
        self.assertEqual(u.name, "Ярослав")
        with self.assertRaises(ValueError):
            u.id = -1
        with self.assertRaises(ValueError):
            u.name = "A"

    def test_currency(self):
        c = Currency(None, None, "USD", "Dollar", 92.5, 1)
        self.assertEqual(c.char_code, "USD")
        self.assertEqual(c.value, 92.5)
        self.assertEqual(c.nominal, 1)
        with self.assertRaises(ValueError):
            c.value = -1
        with self.assertRaises(ValueError):
            c.char_code = "US"

# 2. Тестирование функции get_currencies
class TestGetCurrencies(unittest.TestCase):

    def test_get_currencies_live(self):
        # Берём реальные курсы с сайта ЦБ РФ
        codes = ['USD', 'EUR', 'GBP']
        result = get_currencies(codes)
        for code in codes:
            self.assertIn(code, result)
            self.assertIsInstance(result[code], float)

# 3. Тестирование шаблонов Jinja2
class TestTemplates(unittest.TestCase):

    def setUp(self):
        # Инициализация Jinja2
        self.env = Environment(
            loader=PackageLoader('myapp'),
            autoescape=select_autoescape()
        )

        # Модели для шаблонов
        self.main_author = Author('Владислав Иголкин (504623)', 'P3124')
        self.app_info = App('Клиент-серверное приложение на Python', '1.0',
self.main_author)
        self.users = [
            User(1, 'Ярослав'),
            User(2, 'Виктория'),
            User(3, 'Антон')
        ]
        self.currencies = [
            Currency(None, None, "USD", "Dollar", 92.5, 1),
            Currency(None, None, "EUR", "Euro", 101.2, 1),
            Currency(None, None, "GBP", "Pound", 118.3, 1)

```

```

]

def test_index_template(self):
    template = self.env.get_template('index.html')
    html = template.render(
        myapp=self.app.info.name,
        author_name=self.main_author.name,
        group=self.main_author.group,
        navigation=[{'caption': 'Главная', 'href': '/'}]
    )
    self.assertIn('Владислав Иголкин', html)
    self.assertIn('Клиент-серверное приложение на Python', html)

def test_users_template(self):
    template = self.env.get_template('users.html')
    html = template.render(users=self.users)
    self.assertIn('Ярослав', html)
    self.assertIn('Виктория', html)
    self.assertIn('Антон', html)

def test_currencies_template(self):
    template = self.env.get_template('currencies.html')
    html = template.render(currencies=self.currencies)
    self.assertIn('USD', html)
    self.assertIn('EUR', html)
    self.assertIn('GBP', html)

# Запуск тестов
if __name__ == "__main__":
    unittest.main()

```

```

C:\Users\solik\PycharmProjects\lab8\.venv\Scripts
Testing started at 19:34 ...
Launching unittests with arguments python -m un

Ran 7 tests in 0.249s

OK

Process finished with exit code 0

```

8. Выводы:

Какие проблемы возникли

- Трудно было тестировать контроллер, потому что HTTPServer рассчитан на настоящие подключения, а не на тестовые объекты.
- Были сложности с кодировкой русских символов в курсах валют — их нельзя было просто вставить в байтовую строку.

Как я применил принципы MVC:

- Модели (User, Currency, Author, App) хранят данные и проверки, но не показывают их на экране.
- Контроллер обрабатывает запросы пользователя и решает, что показать.

- Шаблоны Jinja2 (Views) формируют HTML, чтобы пользователь видел страницы.
- Благодаря этому я мог менять шаблоны или данные отдельно, и всё работало.

Что нового я узнал

- Как работать с HTTPServer и обрабатывать маршруты /, /users, /currencies.
- Как использовать Jinja2, передавать данные в шаблоны и создавать HTML страницы.
- Как получать актуальные курсы валют через API ЦБ РФ и обрабатывать XML.
- Как тестировать модели, функции и шаблоны, не подключаясь к реальному серверу.